Halokazi Database Guide

The 30 core tables, explained in plain language

Web-Based CV Builder, Career & Job Opportunity Platform · Halotel Software Department
Database: Oracle 19c · Source of truth: db/halokazi.dbml and db/oracle/*.sql
22 October 2026

How to read this guide

Halokazi's database has 55 tables. This guide explains the **30 that matter most**, in an order where each table builds on the ones before it. The other 25 (lookup lists, sign-in security, extra CV sections, alert preferences) get one line each in Appendix A.

Every table page has the same parts:

- **In one sentence**: what the table is for.
- **Think of it as**: a real-world comparison.
- **In DBML**: the table as it appears in `halokazi.dbml`. Some less important columns are hidden behind `...`.
- **Column groups**: the columns explained in groups, with the requirement each one serves (e.g. APP-01).
- **Example rows, rules and notes** where they help.
- **Connects to**: the tables it links with.

Each part ends with a few questions to check yourself. Answers are in Appendix B.

DBML cheat sheet

You see	It means
`[pk]`	Primary key: the column that uniquely identifies each row.
`[pk]` on two columns	Composite key: the **pair** must be unique (used by junction tables).
`[increment]`	The number is created automatically: 1, 2, 3...
`[not null]`	A value is required.
`[unique]`	No two rows may share this value.
`[default: 0]`	Value used when none is given.
`ref: > other.id`	Many-to-one: many rows here point to one row there.
`ref: - other.id`	One-to-one: one row here, one row there.
`indexes { (a, b) [unique] }`	The combination of a and b may appear only once.
`bool`	Stored as NUMBER(1) in Oracle: 0 = no, 1 = yes.
`json`	Stored as CLOB with an IS JSON check (structured data in one column).
`timestamptz`	Date and time with time zone; shown to users in East Africa Time.

Six patterns you will see again and again

Pattern	What it is	Example
Soft delete	Rows are never erased; a `deleted_at` date is filled in instead, so history stays intact.	`app_user`, `cv`, `vacancy`
Snapshot	A frozen copy taken at an important moment, so later edits do not change what was sent.	`application`
Lookup table	A small managed list that other tables point to, so everyone uses the same values.	`skill`, `region`
Junction table	A table in the middle of a many-to-many link, often carrying extra data.	`profile_skill`
Append-only history	Rows are added automatically and can never be edited or deleted.	`audit_log`, `application_status_history`
Locked after approval	Once someone has checked or relied on a value, it cannot change.	`employer.tin`, vacancy title

The 30 tables at a glance

#	Table	Part	In one sentence
1	app_user	Who uses the system	Every person who can log in to Halokazi has exactly one row here, whatever their role.
2	job_seeker_profile	Who uses the system	Extra information only job seekers need: contact details, address, summary, photo and the completion score.
3	employer	Who uses the system	The company behind an employer login: its official identity, current verification status and public profile.
4	vacancy	Jobs and applications	One row per job advert an employer creates, from draft until it closes.
5	cv	Jobs and applications	A design choice, not a copy of the data: 'show my profile using this template, with these sections'.
6	application	Jobs and applications	One row each time a job seeker applies to a vacancy. Where the job seeker side and the employer side meet.
7	application_status_history	Jobs and applications	A permanent timeline of every status an application has had.
8	application_certificate	Jobs and applications	Which certificates a job seeker attached to a particular application, with a frozen copy of their details.
9	education	Job seeker profile details	Each school, college or university a job seeker attended. One row per qualification.
10	work_experience	Job seeker profile details	Each job a job seeker has held. One row per job.
11	skill	Job seeker profile details	The master list of skills everyone picks from: Excel, Customer Care, Oracle SQL, Driving...
12	profile_skill	Job seeker profile details	Which skills each job seeker has, and how good they are at each.
13	certification	Job seeker profile details	Professional certificates a job seeker holds (CPA, CCNA, First Aid...), with the scanned file.
14	spoken_language	Job seeker profile details	The list of languages a person can say they speak: Swahili, English, French, Arabic...
15	profile_language	Job seeker profile details	Which languages each job seeker speaks, and how well.
16	profile_section	Job seeker profile details	The nine profile sections and their weights. This table is the recipe for the completion %.
17	profile_section_na	Job seeker profile details	The sections a job seeker has marked 'Not Applicable'.
18	cv_template	CV design	The CV designs available to choose from. Five come pre-loaded.
19	cv_section	CV design	For each CV: which sections are shown, in what order, and with what heading.
20	employer_verification	Employer verification and vacancy lifecycle	Every verification request an employer submits, and the admin's decision on it.
21	employer_verification_doc	Employer verification and vacancy lifecycle	The uploaded documents that belong to a verification submission.
22	vacancy_skill	Employer verification and vacancy lifecycle	Which skills a vacancy asks for.

#	Table	Part	In one sentence
23	vacancy_status_history	Employer verification and vacancy lifecycle	A permanent timeline of every status a vacancy has had.
24	vacancy_reopen_request	Employer verification and vacancy lifecycle	An employer's request to reopen a closed vacancy, and the admin's answer.
25	saved_vacancy	Employer verification and vacancy lifecycle	Jobs a seeker has bookmarked to look at later.
26	admin_permission	Admin and accountability	The list of individual powers an Admin can be given.
27	admin_permission_assignment	Admin and accountability	Which Admin holds which permission, who gave it, and if and when it was taken back.
28	audit_log	Admin and accountability	A permanent record of every important action taken in the system, by anyone.
29	notification	Notifications and files	The in-app messages a user sees under the bell icon, with read/unread status.
30	stored_file	Notifications and files	The record of every file uploaded or generated: profile photos, certificates, verification documents, CV PDFs, report exports.

Part 1 · Who uses the system

Every person who logs in, and the two kinds of 'extra details' a person can have.

Table 1 · app_user

Who uses the system

Every person who can log in to Halokazi has exactly one row here, whatever their role.

> **Think of it as:** *The ID card at the gate. It says who you are and whether you are allowed in. It does not hold your CV or your company details; those live in other tables.*

IN DBML

```
Table app_user {
  user_id number [pk, increment]
  full_name varchar2(150) [not null]
  email varchar2(254) [not null]          // unique, any capitals
  phone varchar2(20) [not null, unique]
  password_hash varchar2(255) [not null]
  role varchar2(20) [not null]
  account_status varchar2(30) [not null]
  phone_verified bool [not null, default: 0]
  created_by number [ref: > app_user.user_id]
  suspended_by number [ref: > app_user.user_id]
  deleted_at timestamptz
  ...
}
```

Identity: who you are

- `user_id` is your unique number, created automatically (`increment`).
- `full_name`, `email`, `phone` come from the registration form (AUTH-01).
- `email` and `phone` are **unique** because you can log in with either one (AUTH-04).

Login: proving it is you

- `password_hash` is a scrambled version of the password, never the password itself. If the database ever leaks, real passwords do not.
- `failed_login_count` and `locked_until` lock the account for a while after too many wrong passwords (AUTH-13).

Role and status

- `role` is one of JOB_SEEKER, EMPLOYER, ADMIN or SUPER_ADMIN. This one column decides which screens you see.
- `account_status`: PENDING_VERIFICATION (phone not verified yet), ACTIVE, SUSPENDED, DEACTIVATED, DELETED.
- `suspended_by` and `suspension_reason` record who blocked the account and why (AUTH-10).

Soft delete

- When someone deletes their account the row is **not erased**. `deleted_at` and `anonymized_at` are filled in and personal details are blanked, so their old applications still make sense to employers (AUTH-11).

EXAMPLE ROWS

user_id	full_name	role	account_status	created_by
1	Neema Mushi	SUPER_ADMIN	ACTIVE	
2	Asha Juma	JOB_SEEKER	ACTIVE	
3	John (Baraka Ltd)	EMPLOYER	ACTIVE	
4	Peter Mollel	ADMIN	ACTIVE	1

A table pointing at itself. `created_by` and `suspended_by` point back to `app_user`. Peter's `created_by = 1` means Neema, the Super Admin, created his account.

Connects to: Parent of almost everything. Gets one `job_seeker_profile` or one `employer` depending on `role`.

Table 2 · job_seeker_profile

Who uses the system

Extra information only job seekers need: contact details, address, summary, photo and the completion score.

Think of it as: *The front page of a job seeker's file. The detailed sections (education, experience, skills...) are separate pages clipped to it.*

IN DBML

```
Table job_seeker_profile {
  profile_id number [pk, increment]
  user_id number [not null, unique, ref: - app_user.user_id]
  contact_email varchar2(254)
  region_id number [ref: > region.region_id]
  district_id number [ref: > district.district_id]
  professional_summary clob
  profile_picture_file_id number [ref: > stored_file.file_id]
  receive_job_alerts bool [not null, default: 1]
  completion_percentage number(5,2) [not null, default: 0]
  ...
}
```

Why a separate table?

- Employers and admins have no summary or profile photo. Putting these columns on `app_user` would leave them empty for most users, so job-seeker-only data gets its own table.
- `ref: -` means **one-to-one**, and `unique` on `user_id` enforces it: one user, at most one profile.

Columns matched to the profile sections (Requirements section 5)

- Contact Information (10%): `contact_email`, `alternate_phone`, plus email/phone on `app_user`.
- Address/Location (10%): `region_id`, `district_id`, `address_line`.
- Professional Summary (10%): `headline`, `professional_summary`.
- Profile Picture (5%): `profile_picture_file_id`.
- The other 65% (education, experience, skills, certifications, languages) live in separate tables because a person can have **many** of each (Part 3).

Two special columns

- `completion_percentage` is the score shown on screen. When someone applies, the database **recalculates it live** instead of trusting this saved number, so nobody can bypass the 75% rule.
- `receive_job_alerts` decides whether they get 'new matching job' notifications.

Why `region_id` and not a text box? It points to the `region` list, so 'Dar es Salaam', 'Dar' and 'DSM' all become the same region. Search and matching stay reliable.

Connects to: Owned by `app_user` (1:1). Parent of education, work_experience, profile_skill, certification, profile_language, cv, application, saved_vacancy.

Table 3 · employer

Who uses the system

The company behind an employer login: its official identity, current verification status and public profile.

> **Think of it as:** *A company's registration certificate pinned next to its shop sign. The certificate part is locked once checked; the shop sign can be repainted.*

IN DBML

```
Table employer {
  employer_id number [pk, increment]
  user_id number [not null, unique, ref: - app_user.user_id]
  business_name varchar2(200) [not null]
  registration_number varchar2(60) [unique]
  tin varchar2(20) [unique]
  business_licence_number varchar2(60)
  verification_status varchar2(20) [not null]
  approved_at timestamptz
  company_description clob
  website varchar2(300)
  logo_file_id number [ref: > stored_file.file_id]
  ...
}
```

Official identity: locked after approval

- `business_name`, `registration_number`, `tin`, `business_licence_number`.
- `tin` is **unique**: one company, one employer account (VER-11).
- After an admin approves, these four fields are **locked** by a database trigger (VER-10). Otherwise a verified company could quietly turn into a different one.

Verification: can they post jobs?

- `verification_status`: NOT_SUBMITTED, PENDING_REVIEW, APPROVED or REJECTED.
- Only APPROVED employers can create vacancies (AUTH-08, VAC-01).
- This table keeps only the **current** status. The full history of submissions lives in `employer_verification` (table 20).

Public company profile: freely editable

- `company_description`, `website`, `logo_file_id`, `region_id`, `physical_address`, `company_size`, `category_id` (industry), and the `representative_*` contact person.

EXAMPLE ROWS

Moment	verification_status	Can post jobs?
Just registered	NOT_SUBMITTED	No
Uploaded documents	PENDING_REVIEW	No
Admin rejected (missing licence)	REJECTED	No - fix and resubmit
Resubmitted	PENDING_REVIEW	No
Admin approved	APPROVED	Yes

> **Rule of thumb:** if an admin checked it, it is locked. If nobody checked it, it is free to edit.

Connects to: Owned by `app_user` (1:1). Parent of `employer_verification` and `vacancy`.

Check your understanding · Part 1

1. Asha changes the email address she logs in with. Which table changes?
2. Why can an employer change its website after approval, but not its TIN?
3. Peter is an Admin. Does he have a row in `job_seeker_profile`? In `employer`?

Part 2 · Jobs and applications

The heart of Halokazi: the job advert, the CV and the application where they meet.

Table 4 · vacancy

Jobs and applications

One row per job advert an employer creates, from draft until it closes.

> **Think of it as:** *The advert pinned on the notice board, with a sticker showing whether it is still open.*

IN DBML

```
Table vacancy {
  vacancy_id number [pk, increment]
  employer_id number [not null, ref: > employer.employer_id]
  job_title varchar2(200) [not null]
  job_description clob [not null]
  job_requirements clob [not null]
  qualifications clob [not null]
  work_experience varchar2(500) [not null]
  min_experience_years number(4,1) [not null, default: 0]
  region_id number [not null, ref: > region.region_id]
  employment_type_id number [not null, ref: > employment_type.employment_type_id]
  category_id number [ref: > job_category.category_id]
  number_of_positions number [not null]
  application_deadline timestamptz [not null]
  salary_min number(14,2)
  status varchar2(20) [not null]
  draft_expires_at timestamptz
  ...
}
```

What the job is (mandatory, Requirements section 8)

- `job_title`, `job_description`, `job_requirements`, `qualifications`, `work_experience`.
- `number_of_positions` must be greater than 0, and `application_deadline` must be in the future (VAC-09).

Structured fields for search and matching

- `region_id`, `district_id`, `employment_type_id`, `category_id`, `min_education_level_id`, `min_experience_years`.
- Why both `work_experience` (text) and `min_experience_years` (number)? The text is for **people** ('3+ years in telecom billing'). The number is for **the computer** ('does Asha have 3 or more years?').

Salary (optional)

- `salary_min`, `salary_max`, `salary_currency` (TZS by default), `salary_period`.

Lifecycle

- `status` moves DRAFT, ACTIVE, CLOSED, with side routes to EXPIRED (drafts older than 14 days, VAC-02) and DELETED (soft delete).
- `published_at`, `closed_at`, `closure_reason` (DEADLINE_REACHED, MANUAL, ADMIN_ACTION) record what happened and when.
- A scheduled job closes vacancies at the deadline every 5 minutes (VAC-06). Late applications are also blocked directly, even between runs.

- CLOSED back to ACTIVE needs an approved `vacancy_reopen_request` (table 24).

RULES ON THIS TABLE

Rule	Why
Once anyone has applied, title, requirements, qualifications, education, experience, employment type and region are locked	VAC-04: applicants must not be judged against a job they did not apply for
Only an APPROVED, ACTIVE employer can create or publish	VAC-01, AUTH-08
Deleting keeps the row (soft delete)	VAC-07: existing applications stay viewable

Connects to: Belongs to `employer`. Parent of `application`, `vacancy_skill`, `vacancy_status_history`, `vacancy_reopen_request`, `saved_vacancy`.

Table 5 · cv

Jobs and applications

A design choice, not a copy of the data: 'show my profile using this template, with these sections'.

> **Think of it as:** *A frame you choose for one photo. Change the photo (your profile) and every frame shows the new photo.*

IN DBML

```
Table cv {
  cv_id number [pk, increment]
  profile_id number [not null, ref: > job_seeker_profile.profile_id]
  template_id number [not null, ref: > cv_template.template_id]
  cv_name varchar2(100) [not null]
  headline varchar2(150)
  summary_override clob
  cv_language varchar2(2) [not null, default: 'EN']
  latest_pdf_file_id number [ref: > stored_file.file_id]
  deleted_at timestamptz
  ...
}
```

The key idea

- The `cv` table does **not** store education, experience or skills. Those live once, in the profile tables. A CV only chooses how to display them.
- So when Asha adds a certificate to her profile, **all** her CVs show it automatically. If each CV kept its own copy, they would drift apart.

What a CV can customise

- `template_id`: which of the 5 designs to use (CV-02).
- `headline` and `summary_override`: a pitch tailored to this CV. Empty means 'use my profile summary'.
- `cv_language`: EN or SW.
- Which sections appear, and in what order, is stored in `cv_section` (table 19).
- `latest_pdf_file_id` is the last PDF generated for download (CV-05).

Relationships

- Many CVs to one profile (CV-01: multiple CVs) and many CVs to one template. Both are `ref: >`.
- `deleted_at` is a soft delete, because an application may still point to this CV.

EXAMPLE ROWS

cv_name	template	headline
Accounting CV	Professional	Certified Accountant, 4 years' experience
Banking CV	Modern	Finance & Banking Specialist
Short CV	Minimal	(empty - uses profile summary)

Connects to: Belongs to `job_seeker_profile` and `cv_template`. Has `cv_section` rows. Used by `application`.

Table 6 · application

Jobs and applications

One row each time a job seeker applies to a vacancy. Where the job seeker side and the employer side meet.

> **Think of it as:** *The envelope handed in at the office: a photocopy of your CV inside, and a stamp on the outside showing how far it has got.*

IN DBML

```
Table application {
  application_id number [pk, increment]
  application_ref varchar2(20) [not null, unique]     // HK-2026-000123
  vacancy_id number [not null, ref: > vacancy.vacancy_id]
  profile_id number [not null, ref: > job_seeker_profile.profile_id]
  cv_id number [not null, ref: > cv.cv_id]
  cover_letter clob [not null]
  requirements_confirmed bool [not null]
  profile_snapshot json [not null]
  cv_snapshot json [not null]
  cv_snapshot_file_id number [not null, ref: > stored_file.file_id]
  profile_completion_at_submit number(5,2) [not null]
  status varchar2(20) [not null]
  status_changed_by number [ref: > app_user.user_id]
  ...
}
```

Who, for what, with which CV

- Three `ref: >` links: `profile_id` (who), `vacancy_id` (for what), `cv_id` (with which CV, exactly one: CV-07).
- `application_ref` is a friendly number for screens and support calls.

What the seeker submits

- `cover_letter` is required (project decision; note this differs from APP-04).
- `requirements_confirmed` is the 'I meet the requirements' tick box and must be 1.
- Attached certificates go in `application_certificate` (table 8).

The snapshot: a photocopy taken at submission

- A CV is live; it always shows today's profile. But the employer must see **what was sent** (PROF-04, CV-09, APP-13).
- So at submission the system freezes `profile_snapshot` and `cv_snapshot` (data as JSON), `cv_snapshot_file_id` (the exact PDF) and `profile_completion_at_submit`.
- The database blocks any later change to these columns.

Status: the only part that keeps changing

- SUBMITTED, then IN_REVIEW, then either SHORTLISTED or REJECTED (APP-11). No other moves are allowed.
- Each change is copied automatically to `application_status_history` (table 7).

RULES ON THIS TABLE

Rule	Why
Profile must be at least 75% complete, recalculated live	APP-01
Vacancy must be ACTIVE and before its deadline; employer not suspended	APP-02, VAC-08, VAC-11
Only one non-rejected application per seeker per vacancy	APP-05; reapply after REJECTED is allowed (APP-06)
Editable only while SUBMITTED and before the deadline	APP-07, APP-08
Can never be deleted	APP-09: no withdrawal

Connects to: Belongs to `vacancy`, `job_seeker_profile` and `cv`. Parent of `application_status_history` and `application_certificate`.

Table 7 · application_status_history

Jobs and applications

A permanent timeline of every status an application has had.

> **Think of it as:** *The stamps on a passport. New stamps get added; old ones can never be rubbed out.*

IN DBML

```
Table application_status_history {
  history_id number [pk, increment]
  application_id number [not null, ref: > application.application_id]
  from_status varchar2(20)
  to_status varchar2(20) [not null]
  changed_by number [ref: > app_user.user_id]
  changed_at timestamptz [not null]
  remarks varchar2(1000)
}
```

How rows get here

- Nobody inserts them by hand. A database trigger writes a row **automatically** whenever an application is created or its status changes.
- The first row has an empty `from_status` and `to_status = SUBMITTED`, with the applicant as `changed_by`.

Why it exists

- APP-12: every status change must be recorded with date and time and stay immutable.
- Another trigger blocks every UPDATE and DELETE, so the timeline cannot be rewritten, even by a developer mistake.

EXAMPLE ROWS

from_status	to_status	changed_by	changed_at
	SUBMITTED	Asha	1 Oct 09:14
SUBMITTED	IN_REVIEW	John (Baraka Ltd)	4 Oct 11:02
IN_REVIEW	SHORTLISTED	John (Baraka Ltd)	9 Oct 15:40

Connects to: Many rows to one `application`.

Table 8 · application_certificate

Jobs and applications

Which certificates a job seeker attached to a particular application, with a frozen copy of their details.

> **Think of it as:** *Photocopies of your certificates stapled to the application form.*

IN DBML

```
Table application_certificate {
  application_id number [pk, ref: > application.application_id]
  certification_id number [pk, ref: > certification.certification_id]
  certification_name varchar2(200) [not null]
  issuing_organization varchar2(200) [not null]
  issue_date date
  expiry_date date
  certificate_file_id number [ref: > stored_file.file_id]
  attached_at timestamptz [not null]
}
```

A junction table with extra data

- It links many applications to many certificates. The two `[pk]` columns together are the key, so the same certificate cannot be attached twice to one application.
- It also stores a **copy** of the name, organisation, dates and file, filled in automatically. If Asha later edits or removes the certificate from her profile, the application still shows what she sent.

Rules

- Only the applicant's own, non-deleted certificates can be attached.
- Attach or remove only while the application is SUBMITTED and before the deadline; after that it is locked.

Connects to: Joins `application` and `certification`.

Check your understanding · Part 2

1. Asha adds a new skill to her profile. Does she have to update each of her 3 CVs?
2. Asha was REJECTED and the vacancy is still open. Can she apply again? What if she was SHORTLISTED?
3. Baraka Ltd has 5 applications and wants to rename 'Accountant' to 'Senior Accountant'. What happens?

Part 3 · Job seeker profile details

Everything that makes up a job seeker's profile, and how the completion % is worked out.

Table 9 · education

Job seeker profile details

Each school, college or university a job seeker attended. One row per qualification.

> **Think of it as:** *One line in the 'Education' section of a CV.*

IN DBML

```
Table education {
  education_id number [pk, increment]
  profile_id number [not null, ref: > job_seeker_profile.profile_id]
  education_level_id number [not null, ref: > education_level.education_level_id]
  institution_name varchar2(200) [not null]
  qualification varchar2(200) [not null]
  field_of_study varchar2(150)
  grade varchar2(50)
  start_date date
  end_date date
  is_current bool [not null, default: 0]
}
```

One-to-many

- `ref: >` to the profile: one person can have many education rows (O-Level, Diploma, Degree...).

Two ways to describe the qualification

- `qualification` is the exact award as a person writes it: 'BSc Computer Science'.
- `education_level_id` points to an ordered list (Primary, O-Level, A-Level, Certificate, Diploma, Bachelor, Master, PhD). The computer can then answer 'is this at least a Bachelor?' when matching against a vacancy's minimum education.

Small checks

- `end_date` cannot be before `start_date`. `is_current` marks studies still in progress.
- Having at least one row completes the Education section (15%).

Connects to: Many rows to one `job_seeker_profile`. Uses the `education_level` list.

Table 10 · work_experience

Job seeker profile details

Each job a job seeker has held. One row per job.

> **Think of it as:** *One entry in the 'Work Experience' section of a CV.*

IN DBML

```
Table work_experience {
  work_experience_id number [pk, increment]
  profile_id number [not null, ref: > job_seeker_profile.profile_id]
  company_name varchar2(200) [not null]
  job_title varchar2(150) [not null]
  employment_type_id number [ref: > employment_type.employment_type_id]
  location varchar2(150)
  description clob
  start_date date [not null]
  end_date date
  is_current bool [not null, default: 0]
}
```

Columns

- `company_name`, `job_title`, `description`: what a reader sees on the CV.
- `employment_type_id`: full-time, contract, internship... from the shared list.
- `start_date` is required; `end_date` may be empty only if `is_current = 1` (they still work there).

Not Applicable

- Fresh graduates have no jobs yet. This section is allowed to be marked Not Applicable, so they are not penalised (see table 17).

Connects to: Many rows to one `job_seeker_profile`.

Table 11 · skill

Job seeker profile details

The master list of skills everyone picks from: Excel, Customer Care, Oracle SQL, Driving...

> **Think of it as:** *A dictionary of skills. People do not write their own words; they point to an entry in the dictionary.*

IN DBML

```
Table skill {
  skill_id number [pk, increment]
  name_en varchar2(100) [not null]     // unique, ignoring capitals
  name_sw varchar2(100)
  category_id number [ref: > job_category.category_id]
  is_active bool [not null, default: 1]
}
```

Why a list and not free text?

- If people typed skills freely, 'Excel', 'MS Excel' and 'excel' would become three different skills, and matching a vacancy needing 'Excel' would miss two of them.
- `name_en` is unique ignoring capitals, so 'EXCEL' cannot be added as a duplicate.

Other columns

- `name_sw` gives the Swahili name.
- `category_id` groups skills by industry, which helps suggest relevant skills.
- `is_active = 0` retires a skill without breaking people who already have it.

Connects to: Used by `profile_skill` (job seekers) and `vacancy_skill` (vacancies). Maintained by admins with the reference-data permission.

Table 12 · profile_skill

Job seeker profile details

Which skills each job seeker has, and how good they are at each.

> **Think of it as:** *Ticking skills in the dictionary and writing your level next to each tick.*

IN DBML

```
Table profile_skill {
  profile_id number [pk, ref: > job_seeker_profile.profile_id]
  skill_id number [pk, ref: > skill.skill_id]
  proficiency varchar2(20) [not null]
  years_experience number(4,1)
}
```

The junction-table pattern

- One seeker has many skills; one skill belongs to many seekers. That is **many-to-many**, so a table sits in the middle.
- The two `[pk]` columns together are the key: the same person cannot list the same skill twice.

Extra data on the link

- `proficiency`: BEGINNER, INTERMEDIATE, ADVANCED or EXPERT. `years_experience` is optional.
- This is why a junction table beats a plain `<>` relationship: the link itself can carry information.

EXAMPLE ROWS

profile	skill	proficiency
Asha	Excel	ADVANCED
Asha	Customer Care	EXPERT
Juma	Excel	BEGINNER

Connects to: Joins `job_seeker_profile` and `skill`. Compared with `vacancy_skill` for matching.

Table 13 · certification

Job seeker profile details

Professional certificates a job seeker holds (CPA, CCNA, First Aid…), with the scanned file.

> **Think of it as:** *Your folder of certificates, each with a scanned copy.*

IN DBML

```
Table certification {
  certification_id number [pk, increment]
  profile_id number [not null, ref: > job_seeker_profile.profile_id]
  certification_name varchar2(200) [not null]
  issuing_organization varchar2(200) [not null]
  credential_id varchar2(100)
  issue_date date
  expiry_date date
  certificate_file_id number [ref: > stored_file.file_id]
  deleted_at timestamptz
}
```

Columns

- `certification_name`, `issuing_organization`, `credential_id` (the number printed on the certificate).
- `expiry_date` cannot be before `issue_date`.
- `certificate_file_id` points to the uploaded scan in `stored_file`.

Soft delete

- Certificates can be attached to applications, so removing one only sets `deleted_at`. The application keeps its own copy anyway (table 8).

Connects to: Many rows to one `job_seeker_profile`. Can be attached through `application_certificate`.

Table 14 · spoken_language

Job seeker profile details

The list of languages a person can say they speak: Swahili, English, French, Arabic...

> **Think of it as:** *The language dropdown.*

IN DBML

```
Table spoken_language {
  language_id number [pk, increment]
  iso_code varchar2(10) [not null, unique]    // 'sw', 'en', 'fr'
  name_en varchar2(60) [not null]
  name_sw varchar2(60) [not null]
  is_active bool [not null, default: 1]
}
```

Notes

- Named `spoken_language` because LANGUAGE is a reserved word in Oracle.
- `iso_code` is the international short code. `name_en` and `name_sw` show the name in the user's interface language: 'Swahili' or 'Kiswahili'.
- This is a **lookup table**: a small, managed list other tables point to.

Connects to: Used by `profile_language`.

Table 15 · profile_language

Job seeker profile details

Which languages each job seeker speaks, and how well.

> **Think of it as:** *The 'Languages' section of a CV.*

IN DBML

```
Table profile_language {
  profile_id number [pk, ref: > job_seeker_profile.profile_id]
  language_id number [pk, ref: > spoken_language.language_id]
  proficiency varchar2(20) [not null]
}
```

Same pattern as profile_skill

- A junction table between the profile and the language list, with a composite key so a language is listed once per person.
- `proficiency`: BASIC, CONVERSATIONAL, FLUENT or NATIVE.
- At least one row completes the Languages section (10%).

Connects to: Joins `job_seeker_profile` and `spoken_language`.

Table 16 · profile_section

Job seeker profile details

The nine profile sections and their weights. This table is the recipe for the completion %.

> **Think of it as:** *The marking scheme for an exam: how many marks each question is worth.*

IN DBML

```
Table profile_section {
  section_code varchar2(30) [pk]
  name_en varchar2(80) [not null]
  name_sw varchar2(80) [not null]
  weight number(5,2) [not null]
  can_be_na bool [not null, default: 0]
  sort_order number [not null]
  is_active bool [not null, default: 1]
}
```

Why weights live in a table

- The Requirements fix the weights (Education 15%, Profile Picture 5%...). Storing them as data means management can change a weight later **without** changing code.

How the score is calculated

- A database function, `hk_profile_completion`, adds up the `weight` of every section the seeker has completed.
- A section also counts as complete if it is marked Not Applicable **and** `can_be_na = 1` for that section.

EXAMPLE ROWS

section_code	weight	can_be_na
CONTACT	10	No
EDUCATION	15	No
WORK_EXPERIENCE	15	Yes
SKILLS	15	No
CERTIFICATIONS	10	Yes
LANGUAGES	10	No
ADDRESS	10	No
SUMMARY	10	No
PROFILE_PICTURE	5	No

Here the primary key is a **readable code** (`section_code`) instead of a number. That is fine for small fixed lists and makes other rows easier to read.

Connects to: Used by `profile_section_na` and the completion function.

Table 17 · profile_section_na

Job seeker profile details

The sections a job seeker has marked 'Not Applicable'.

> **Think of it as:** *Writing 'N/A' next to a question on a form, so it is not counted against you.*

IN DBML

```
Table profile_section_na {
  profile_id number [pk, ref: > job_seeker_profile.profile_id]
  section_code varchar2(30) [pk, ref: > profile_section.section_code]
  marked_at timestamptz [not null]
}
```

Why it exists

- The Requirements say people should not be penalised for information they genuinely do not have, such as a fresh graduate with no work experience.
- A row here means 'treat this section as complete for me'.

The safety catch

- Marking a section N/A only counts when `profile_section.can_be_na = 1`. Right now that is Work Experience and Certifications. Marking Skills as N/A would simply not count.

> **Open question:** cover letter and certificates are required on applications. If someone marks Certifications N/A, should they be able to apply? This still needs a decision.

Connects to: Joins `job_seeker_profile` and `profile_section`.

Check your understanding · Part 3

1. Why is `skill` a separate table instead of a text column on the profile?
2. A fresh graduate has no work experience. How do they still reach 75%?
3. Where would you change the weight of 'Profile Picture' from 5% to 10%?

Part 4 · CV design

How a CV decides what it looks like, without copying any profile data.

Table 18 · cv_template

CV design

The CV designs available to choose from. Five come pre-loaded.

> **Think of it as:** *The design gallery in a word processor: Classic, Modern, Minimal...*

IN DBML

```
Table cv_template {
  template_id number [pk, increment]
  code varchar2(40) [not null, unique]
  name_en varchar2(100) [not null]
  name_sw varchar2(100) [not null]
  layout_definition clob
  preview_file_id number [ref: > stored_file.file_id]
  version number [not null, default: 1]
  status varchar2(10) [not null]       // ACTIVE | INACTIVE
  created_by number [ref: > app_user.user_id]
  updated_by number [ref: > app_user.user_id]
}
```

Columns

- `code`: a stable name such as CLASSIC or MODERN. Seeded: Classic, Modern, Professional, Minimal, Graduate (CV-02).
- `layout_definition`: the template markup the PDF generator uses. The dev team fills this in.
- `preview_file_id`: a thumbnail picture shown when choosing.

Managing templates (CV-10)

- Super Admin and admins with the CV_TEMPLATE_MANAGE permission maintain them. `created_by` and `updated_by` record who.
- Retire a template with `status = INACTIVE` instead of deleting it, so CVs that already use it keep working.

Connects to: Many `cv` rows point to one template.

Table 19 · cv_section

CV design

For each CV: which sections are shown, in what order, and with what heading.

> **Think of it as:** *The table of contents of one particular CV.*

IN DBML

```
Table cv_section {
  cv_id number [pk, ref: > cv.cv_id]
  section_code varchar2(30) [pk]
  is_visible bool [not null, default: 1]
  display_order number [not null]
  custom_title varchar2(100)
}
```

Columns

- `section_code` is one of: CONTACT, SUMMARY, WORK_EXPERIENCE, EDUCATION, SKILLS, CERTIFICATIONS, LANGUAGES, PROJECTS, ACHIEVEMENTS, REFERENCES, INTERESTS (covers CV-03).
- `is_visible = 0` hides a section on this CV only.
- `display_order` sets the order; `custom_title` renames a heading, e.g. 'Key Achievements'.

Example

- Asha's 'Graduate CV' puts EDUCATION first; her 'Short CV' hides REFERENCES and INTERESTS. Her profile data is the same; only the arrangement differs.

Connects to: Many rows to one `cv`.

Check your understanding · Part 4

1. Asha's 'Short CV' should hide References. Which table records that?
2. An admin deactivates a template that 200 CVs already use. Why is `status` better than deleting the row?

Part 5 · Employer verification and vacancy lifecycle

Proving a company is real, and following a vacancy from draft to closed (and back).

Table 20 · employer_verification

Employer verification and vacancy lifecycle

Every verification request an employer submits, and the admin's decision on it.

> **Think of it as:** *A file of application forms at a registration office, each stamped APPROVED or REJECTED. Old forms are never thrown away.*

IN DBML

```
Table employer_verification {
  verification_id number [pk, increment]
  employer_id number [not null, ref: > employer.employer_id]
  submission_no number [not null]
  business_name varchar2(200) [not null]
  registration_number varchar2(60) [not null]
  tin varchar2(20) [not null]
  business_licence_number varchar2(60) [not null]
  representative_name varchar2(150) [not null]
  status varchar2(20) [not null]
  submitted_by number [not null, ref: > app_user.user_id]
  submitted_at timestamptz [not null]
  reviewed_by number [ref: > app_user.user_id]
  reviewed_at timestamptz
  decision_notes varchar2(2000)
}
```

Why the details are repeated here

- Each row stores the business details **as they were submitted that time**. If the first submission had a wrong TIN and the second a corrected one, both versions are kept (VER-06).
- On approval, the app copies the approved details onto `employer`, where they become locked.

The decision

- `status`: PENDING_REVIEW, APPROVED or REJECTED. `reviewed_by` and `reviewed_at` record who decided and when (VER-07).
- A rejection must include `decision_notes` explaining why, so the employer knows what to fix.

Rules

- Only one PENDING_REVIEW submission per employer at a time.
- Once decided, a row is **final**: a trigger blocks edits and deletes. To try again, the employer submits a new row (VER-05).

EXAMPLE ROWS

submission_no	status	decision_notes
1	REJECTED	Business licence missing
2	REJECTED	TIN does not match certificate
3	APPROVED	

Connects to: Many rows to one `employer`. Parent of `employer_verification_doc`.

Table 21 · employer_verification_doc

Employer verification and vacancy lifecycle

The uploaded documents that belong to a verification submission.

> **Think of it as:** *The attachments stapled to one registration form.*

IN DBML

```
Table employer_verification_doc {
  document_id number [pk, increment]
  verification_id number [not null, ref: > employer_verification.verification_id]
  document_type varchar2(40) [not null]
  file_id number [not null, ref: > stored_file.file_id]
  uploaded_at timestamptz [not null]
  indexes {
    (verification_id, document_type) [unique]
  }
}
```

Columns

- `document_type`: CERTIFICATE_OF_INCORPORATION, TIN_CERTIFICATE, BUSINESS_LICENCE, REPRESENTATIVE_ID or OTHER (VER-01).
- `file_id` points to the actual file in `stored_file`, which tracks the malware scan.

A composite unique index in real life

- `(verification_id, document_type) [unique]`: each submission has at most **one** document of each type. Two business licences on the same submission are refused, but submission #2 can have its own new licence.

Connects to: Many rows to one `employer_verification`. Each points to one `stored_file`.

Table 22 · vacancy_skill

Employer verification and vacancy lifecycle

Which skills a vacancy asks for.

> **Think of it as:** *The 'Required skills' bullet list on a job advert, with each item ticked as must-have or nice-to-have.*

IN DBML

```
Table vacancy_skill {
  vacancy_id number [pk, ref: > vacancy.vacancy_id]
  skill_id number [pk, ref: > skill.skill_id]
  is_mandatory bool [not null, default: 1]
}
```

How it is used

- A junction table between `vacancy` and the same `skill` list job seekers use. Because both sides point at the same list, matching is a simple comparison.
- `is_mandatory` separates must-have skills from nice-to-have ones.
- Rule-based matching (Requirements section 9) counts how many of a vacancy's skills a seeker has in `profile_skill`.

Connects to: Joins `vacancy` and `skill`.

Table 23 · vacancy_status_history

Employer verification and vacancy lifecycle

A permanent timeline of every status a vacancy has had.

> **Think of it as:** *The same passport-stamp idea as application history, but for job adverts.*

IN DBML

```
Table vacancy_status_history {
  history_id number [pk, increment]
  vacancy_id number [not null, ref: > vacancy.vacancy_id]
  from_status varchar2(20)
  to_status varchar2(20) [not null]
  reason varchar2(30)
  changed_by number [ref: > app_user.user_id]    // empty = system
  changed_at timestamptz [not null]
}
```

How it works

- Written automatically by a trigger whenever a vacancy is created or changes status. UPDATE and DELETE are blocked.
- `changed_by` is empty when **the system** made the change, for example the scheduled job closing a vacancy at its deadline.
- `reason` copies the closure reason (DEADLINE_REACHED, MANUAL, ADMIN_ACTION).

Why it is useful

- Answers questions like 'how long was this job open?' and 'who closed it?' in support cases and reports.

EXAMPLE ROWS

from_status	to_status	reason	changed_by
	DRAFT		John
DRAFT	ACTIVE		John
ACTIVE	CLOSED	DEADLINE_REACHED	(system)
CLOSED	ACTIVE		John (after approval)

Connects to: Many rows to one vacancy.

Table 24 · vacancy_reopen_request

Employer verification and vacancy lifecycle

An employer's request to reopen a closed vacancy, and the admin's answer.

> **Think of it as:** *Asking the office manager for permission to reopen a closed counter.*

IN DBML

```
Table vacancy_reopen_request {
  request_id number [pk, increment]
  vacancy_id number [not null, ref: > vacancy.vacancy_id]
  requested_by number [not null, ref: > app_user.user_id]
  reason varchar2(1000) [not null]
  new_deadline timestamptz [not null]
  status varchar2(20) [not null]
  reviewed_by number [ref: > app_user.user_id]
  reviewed_at timestamptz
  review_notes varchar2(1000)
}
```

Why it exists

- VAC-10: a closed vacancy may be reopened only through an employer request approved by an authorised admin. This table was missing from the Stage 5 ERD.

Columns

- `reason` and `new_deadline` come from the employer. `status`: PENDING, APPROVED, REJECTED or CANCELLED.
- `reviewed_by`, `reviewed_at`, `review_notes` record the admin's decision.

Rules

- Only one PENDING request per vacancy at a time.
- The reviewer must be a different person from the requester.
- The vacancy itself refuses to go from CLOSED to ACTIVE unless an APPROVED request exists that was decided after it closed.

Connects to: Many requests to one `vacancy`.

Table 25 · saved_vacancy

Employer verification and vacancy lifecycle

Jobs a seeker has bookmarked to look at later.

> **Think of it as:** *Folding the corner of a page in a newspaper's job section.*

IN DBML

```
Table saved_vacancy {
  profile_id number [pk, ref: > job_seeker_profile.profile_id]
  vacancy_id number [pk, ref: > vacancy.vacancy_id]
  saved_at timestamptz [not null]
}
```

How it works

- A simple junction table: one row means 'this seeker saved this vacancy'. Un-saving deletes the row.
- The two `[pk]` columns make the **pair** unique, so the same job cannot be saved twice by the same person, while many people can save the same job.
- Employer analytics use it to show how many times a vacancy was saved.

Connects to: Joins `job_seeker_profile` and `vacancy`.

Check your understanding · Part 5

1. An employer was rejected twice and approved on the third try. How many `employer_verification` rows exist?
2. Why does `vacancy_status_history` allow `changed_by` to be empty?
3. Can the same employer user approve their own reopen request?

Part 6 · Admin and accountability

Who is allowed to do what, and a permanent record of what everyone did.

Table 26 · admin_permission

Admin and accountability

The list of individual powers an Admin can be given.

> **Think of it as:** *The set of keys hanging in the security office. Each key opens one door.*

IN DBML

```
Table admin_permission {
  permission_id number [pk, increment]
  code varchar2(60) [not null, unique]
  module varchar2(40) [not null]
  name_en varchar2(100) [not null]
  name_sw varchar2(100) [not null]
  description varchar2(400)
}
```

Why individual permissions?

- The Requirements say Admin permissions are assigned **individually** by the Super Admin, and an Admin can only do what they were given (section 3).

The 11 seeded permissions

- EMPLOYER_VERIFICATION_REVIEW, USER_VIEW, USER_SUSPEND, VACANCY_MANAGE, VACANCY_REOPEN_REVIEW, REPORT_MODERATE, ANALYTICS_VIEW, ANALYTICS_EXPORT, AUDIT_LOG_VIEW, CV_TEMPLATE_MANAGE, REFERENCE_DATA_MANAGE.
- `code` is what the app checks; `module` groups them on the admin screen.

Connects to: Handed out through `admin_permission_assignment`.

Table 27 · admin_permission_assignment

Admin and accountability

Which Admin holds which permission, who gave it, and if and when it was taken back.

> **Think of it as:** *The key register: 'Key 4 given to Peter by Neema on 3 Oct; returned 20 Oct'.*

IN DBML

```
Table admin_permission_assignment {
  assignment_id number [pk, increment]
  user_id number [not null, ref: > app_user.user_id]
  permission_id number [not null, ref: > admin_permission.permission_id]
  assigned_by number [not null, ref: > app_user.user_id]
  assigned_at timestamptz [not null]
  revoked_by number [ref: > app_user.user_id]
  revoked_at timestamptz
}
```

How grant and revoke work

- Granting inserts a row. Revoking does **not** delete it; it fills `revoked_by` and `revoked_at`. So there is a full history of who had what, and when.
- A row with an empty `revoked_at` is an active permission. Only one active row per Admin per permission is allowed.

Rules enforced by the database (section 12.1)

- Only a SUPER_ADMIN can grant or revoke.
- Permissions can only be given to users whose role is ADMIN.
- Nobody can grant or revoke their own permissions (`assigned_by` must differ from `user_id`).

EXAMPLE ROWS

Admin	permission	assigned_by	revoked_at
Peter	EMPLOYER_VERIFICATION_REVIEW	Neema	
Peter	AUDIT_LOG_VIEW	Neema	20 Oct
Peter	AUDIT_LOG_VIEW	Neema	(given again later)

Three `ref`s point to `app_user` from one table: the Admin who holds it, the Super Admin who gave it, and the one who revoked it. A table can link to the same table several times for different reasons.

Connects to: Joins `app_user` (Admins) and `admin_permission`.

Table 28 · audit_log

Admin and accountability

A permanent record of every important action taken in the system, by anyone.

> **Think of it as:** *The CCTV recording of the building. Nobody can edit the tape.*

IN DBML

```
Table audit_log {
  audit_id number [pk, increment]
  user_id number [ref: > app_user.user_id]     // empty = system
  actor_role varchar2(20)
  action varchar2(60) [not null]
  entity_type varchar2(40)
  entity_id number
  result varchar2(10) [not null]
  ip_address varchar2(45)
  details json
  created_at timestamptz [not null]
}
```

What each entry records (section 13)

- **Who**: `user_id`, `actor_role`, `ip_address`.
- **Did what**: `action`, e.g. EMPLOYER_APPROVED, USER_SUSPENDED, LOGIN_FAILED.
- **To what**: `entity_type` + `entity_id`, e.g. EMPLOYER + 12.
- **With what result**: `result` is SUCCESS, FAILURE or DENIED (tried something without permission).
- **When**: `created_at`. `details` holds extra facts as JSON (old and new values).

Two design points

- **Append-only**: a trigger blocks every UPDATE and DELETE, so the log can be trusted.
- **No foreign key on** `entity_id`: the log talks about many different tables, so one pair of columns can point at any of them. That is called a polymorphic reference.

EXAMPLE ROWS

user	action	entity	result
Peter	EMPLOYER_APPROVED	EMPLOYER 12	SUCCESS
Peter	AUDIT_LOG_VIEWED	-	DENIED
(system)	VACANCY_AUTO_CLOSED	VACANCY 88	SUCCESS

Connects to: Points to `app_user`. Visible to Super Admin and Admins with AUDIT_LOG_VIEW.

Check your understanding · Part 6

1. Peter tries to give himself the AUDIT_LOG_VIEW permission. What stops him?
2. Why does `audit_log` use `entity_type` + `entity_id` instead of normal foreign keys?

Part 7 · Notifications and files

Messages to users, and how uploaded documents are tracked safely.

Table 29 · notification

Notifications and files

The in-app messages a user sees under the bell icon, with read/unread status.

> **Think of it as:** *Your inbox inside Halokazi.*

IN DBML

```
Table notification {
  notification_id number [pk, increment]
  user_id number [not null, ref: > app_user.user_id]
  template_code varchar2(50) [ref: > notification_template.template_code]
  title varchar2(200) [not null]
  message varchar2(2000) [not null]
  related_entity_type varchar2(30)
  related_entity_id number
  is_read bool [not null, default: 0]
  read_at timestamptz
  created_at timestamptz [not null]
}
```

How a notification is made

- Wording comes from `notification_template`, which has English and Swahili versions with placeholders like {vacancy_title}.
- The app fills in the placeholders in the user's language and saves the **final text** in `title` and `message`. Old notifications keep their wording even if a template changes later.

Columns

- `related_entity_type` + `related_entity_id` say what clicking it opens, e.g. APPLICATION 501.
- `is_read` and `read_at` give read/unread status. History is kept (section 11).

Events that create one

- Application submitted, moved to In Review, rejected, shortlisted; a new relevant vacancy; employer verification approved or rejected; reopen request decided.

Connects to: Many rows to one `app_user`. Uses `notification_template`.

Table 30 · stored_file

Notifications and files

The record of every file uploaded or generated: profile photos, certificates, verification documents, CV PDFs, report exports.

> **Think of it as:** *The register at a storage room's door: what each box is, who owns it, where it sits, and whether security has checked it.*

IN DBML

```
Table stored_file {
  file_id number [pk, increment]
  owner_user_id number [ref: > app_user.user_id]
  file_category varchar2(40) [not null]
  storage_key varchar2(500) [not null, unique]
  original_filename varchar2(255)
  mime_type varchar2(100) [not null]
  size_bytes number [not null]
  checksum_sha256 varchar2(64)
  scan_status varchar2(20) [not null, default: 'PENDING']
  deleted_at timestamptz
}
```

The file itself is not in the database

- The actual bytes live in separate object storage (the Cloud File Storage in the Stage 5 diagrams). This row holds only the description and `storage_key`, the file's address there.
- This keeps the database small and fast, and storage isolated from the app server (section 14.2).

Security columns (section 14.2)

- `mime_type` and `size_bytes` support file-type and size checks.
- `scan_status`: PENDING, CLEAN, INFECTED, FAILED or NOT_REQUIRED. Only CLEAN files should ever be served.
- `checksum_sha256` detects a file that was changed or corrupted.

Who points here

- Profile pictures, certificate scans, verification documents, company logos, CV PDFs, application CV snapshots, template previews and report exports all link to this one table.

Connects to: Pointed to by `job_seeker_profile`, `certification`, `employer`, `employer_verification_doc`, `cv`, `cv_template`, `application`, `application_certificate`, `report_export`.

Check your understanding · Part 7

1. Why are uploaded files kept in object storage instead of inside the database?
2. A certificate upload is still `PENDING` malware scan. Should employers be able to download it?

Appendix A · The other 25 tables

These support the 30 core tables. They are fully defined in `halokazi.dbml` and `01_schema.sql`.

Lookup lists

Table	What it does
`region`	The 31 regions of Tanzania (26 Mainland + 5 Zanzibar), in English and Swahili.
`district`	Districts inside each region. Not seeded yet; load the official list.
`job_category`	Industries and job categories (ICT, Finance, Health...).
`employment_type`	Full-time, part-time, contract, temporary, internship, volunteer.
`education_level`	Ordered education levels from Primary to PhD, used for minimum-education matching.
`report_reason`	Reasons a user can pick when reporting something (fraud, fake vacancy, spam...).
`system_setting`	Business values in one place: 75% threshold, OTP 2 minutes, 10-minute sessions, 14-day drafts...

Security and sign-in

Table	What it does
`otp_code`	SMS verification codes (stored hashed), 2-minute expiry, max 3 resends.
`password_reset_token`	One-time links for 'forgot password' by email.
`user_session`	Logged-in sessions; ended after 10 minutes without activity.
`login_attempt`	Every login attempt, success or failure, for lockouts and rate limits.
`sms_log`	Delivery results of every SMS sent through the SMS gateway.
`account_deletion_request`	A user's request to delete their account, and how it was handled.
`policy_document`	Versions of the Terms, Privacy Policy and consent text.
`policy_acceptance`	Which user accepted which policy version, and when.

Extra profile and CV content

Table	What it does
`profile_reference`	Referees a job seeker lists (name, organisation, contact).
`profile_achievement`	Awards and achievements.
`profile_project`	Projects with role, description and link.
`profile_interest`	Hobbies and interests.

Job-alert preferences

Table	What it does
`profile_preferred_category`	Job categories a seeker wants alerts for.
`profile_preferred_region`	Regions a seeker wants to work in.
`profile_preferred_emp_type`	Employment types a seeker wants (full-time, contract...).

Notifications, moderation and reports

Table	What it does
`notification_template`	English and Swahili wording for each kind of notification.
`user_report`	Reports by users against employers, vacancies or job seekers, and how admins resolved them.
`report_export`	Requests to export reports to Excel or PDF, and the resulting file.

Appendix B · Answers to the questions

Part 1 · Who uses the system

1. **Asha changes the email address she logs in with. Which table changes?**

   `app_user`. (If she only changes the email shown on her CV, that is `job_seeker_profile.contact_email`.)

2. **Why can an employer change its website after approval, but not its TIN?**

   An admin checked the TIN against real documents, so it is locked. Nobody verified the website, so it stays editable.

3. **Peter is an Admin. Does he have a row in `job_seeker_profile`? In `employer`?**

   Neither. Admins only have `app_user` plus rows in `admin_permission_assignment`.

Part 2 · Jobs and applications

1. **Asha adds a new skill to her profile. Does she have to update each of her 3 CVs?**

   No. CVs do not store profile data; they display it. All 3 CVs show the new skill automatically.

2. **Asha was REJECTED and the vacancy is still open. Can she apply again? What if she was SHORTLISTED?**

   After REJECTED: yes, while the vacancy is active. After SHORTLISTED: no, that application still counts as active.

3. **Baraka Ltd has 5 applications and wants to rename 'Accountant' to 'Senior Accountant'. What happens?**

   It is blocked. Job title is a critical field once applications exist, so applicants are never judged against a job they did not apply for.

Part 3 · Job seeker profile details

1. **Why is `skill` a separate table instead of a text column on the profile?**

   So everyone picks from one list. 'Excel', 'MS Excel' and 'excel' do not split into three skills, and matching works.

2. **A fresh graduate has no work experience. How do they still reach 75%?**

   They mark Work Experience as Not Applicable (`profile_section_na`); that section is allowed to be N/A, so it still counts.

3. **Where would you change the weight of 'Profile Picture' from 5% to 10%?**

   In the `profile_section` table (the `weight` column). No code change needed.

Part 4 · CV design

1. **Asha's 'Short CV' should hide References. Which table records that?**

   `cv_section`: the REFERENCES row for that CV with `is_visible = 0`.

2. **An admin deactivates a template that 200 CVs already use. Why is `status` better than deleting the row?**

   Deleting would break the 200 CVs (and old applications) that point to it. INACTIVE just hides it from new CVs.

Part 5 · Employer verification and vacancy lifecycle

1. **An employer was rejected twice and approved on the third try. How many `employer_verification` rows exist?**

   Three. Every submission is kept as history; two are REJECTED and one is APPROVED.

2. **Why does `vacancy_status_history` allow `changed_by` to be empty?**

   Some changes are made by the system itself (the job that closes vacancies at the deadline), not by a person.

3. **Can the same employer user approve their own reopen request?**

   No. A check constraint requires `reviewed_by` to be different from `requested_by`, and only Admins review.

Part 6 · Admin and accountability

1. **Peter tries to give himself the AUDIT_LOG_VIEW permission. What stops him?**

   The database itself: a check constraint forbids `assigned_by = user_id`, and a trigger requires the assigner to be a SUPER_ADMIN.

2. **Why does `audit_log` use `entity_type` + `entity_id` instead of normal foreign keys?**

   The log records actions on many different tables. One pair of columns can point at any of them.

Part 7 · Notifications and files

1. **Why are uploaded files kept in object storage instead of inside the database?**

   Files are large; keeping them outside keeps the database fast and backups small, and storage can be isolated for security.

2. **A certificate upload is still `PENDING` malware scan. Should employers be able to download it?**

   No. Only files whose `scan_status` is CLEAN should be served.